



Namegen – Dokumentaatio

Versio: 0.1.0 (Kehitysvaihe)

Kieli: Puhdas C-kieli

Namegen on komentorivipohjainen sovellus, joka on suunniteltu luomaan **autenttisia suomalaisia nimiä** tietyiltä historiallisilta aikakausilta hyödyntäen vuosikymmenittäin eriteltyä CSV-dataa.



I. Avainominaisuudet ja Käyttö

Namegen tarjoaa vankan ratkaisun historiallisten nimien generointiin komentoriviltä.

- **Historiallinen Datasetsi:** Lukee erillisiä CSV-tiedostoja **etu-, keski- ja sukunimille** miesten ja naisten osalta vuosikymmenittäin eriteltynä.
- **Sukupuolen Valinta:** Tuki sukupuolen valinnalle (-g male / -g female).
- **Aika-kausivalinta:** Komentoriviparametri (-p <numero>) mahdollistaa nimen luonnin valitulta historialliselta jaksolta automaattisesti.
- **Satunnaistettu Rakenne:** Keskinimen käyttö on **satunnaistettu (50% todennäköisyydellä)**.
- **Komentorivikäyttö:** Tukee standardeja CLI-lippuja (-v, -h, -V) vankan virheenkäsittelyn kera.



II. Datarakenteet (Structs)

Ohjelma hyödyntää dynaamiseen muistinhallintaan suunniteltuja rakenteita nimien tallentamiseen ja käsittelyyn.

Rakenne	Kuvaus	Avainkentät	Käyttötarkoitus
NameList	Yksinkertainen lista nimimerkkijonoille.	char **names (nimitaulukko), int count (nimien määrä)	Käytetään sukunimien ja CSV-tiedostojen yksittäisten sarakkeiden tallentamiseen.
DecadeData	Koko CSV-tiedoston säilö, joka käsittelee vuosikymmenittäin jaoteltua dataa.	char **decades (otsikot), NameList *lists (lista NameList-rakenteita), int num_decades (sarakkeiden/vuosikymmenten lkm)	Käytetään etu- ja keskinimien tallentamiseen, joissa jokainen sarake edustaa yhtä vuosikymmentä.



III. Nimien Lataus- ja Muistinhallinta

Ohjelma sisältää kaksi ensisijaista latausfunktiota eri tiedostomuodoille, ja muistinhallinta on toteutettu vapaasti seisovilla vapautusfunktioilla.

Latausfunktiot

- **load_names_multi_column(const char *filename, DecadeData *data)**
 - **Käyttö:** Ladataan monisarakkeiset CSV-tiedostot (etu- ja keskinimet).
 - **Toiminta:** Lukee otsikkorivin ja luo jokaiselle otsikolle uuden NameList-rakenteen (DecadeData.lists). Lukee rivi riviltä ja lisää nimet oikeaan vuosikymmenlistaan sarakkeen perusteella.
- **load_names_simple(const char *filename, NameList *list)**
 - **Käyttö:** Ladataan yksinkertaiset tekstitiedostot (yksi nimi per rivi) tai CSV:n vain ensimmäinen sarake (sukunimet).
 - **Toiminta:** Lukee tiedoston rivi kerrallaan ja tallentaa jokaisen nimen suoraan annettuun NameList-rakenteeseen.

Muistinhallinta

Jokaiselle ladatulle nimelle ja tietorakenteelle varataan muisti **dynaamisesti** (malloc, realloc, strdup). Siksi on välttämätöntä, että muisti vapautetaan:

- **void free_names(NameList *list)**
 - **void free_decade_data(DecadeData *data)**
 - Nämä funktiot käyvät läpi jokaisen varatun merkkijonon ja vapauttavat sen muistin (free()), vapauttavat osoitinlistat ja lopuksi nollaavat laskurit estäen **muistivuodot** (memory leaks).
-



IV. Pääohjelman (main) Logiikka

Vaihe	Toiminto	Tarkoitus
1. Alustus	setlocale(), srand(), Windows-korjaukset	Asettaa UTF-8-lokalisoinnin ja satunnaislukugeneraattorin siemenen.
2. Datat Lataus	Kutsut load_names_multi_column ja load_names_simple	Lataa kaikki etu-, keski- ja sukunimet (miehet ja naiset) muistiin.
3. Komentorivi	Käsittelee liput (-v, -p, -g, -n)	Asettaa generointia ohjaavat muuttujat (esim. period_index, gender_flag, name_count).
4. Tarkistukset	Koko if (first_names.num_decades == 0 ...)	Kriittinen vaihe: Tarkistaa, että pakolliset datatiedostot on ladattu.
5. Nimen Generointi	Silmukka (for (k=0; k < name_count; k++)) ja rand()	Valitsee: a) etunimen valitusta vuosikymmenestä, b) sukunimen koko sukunimilistasta, ja c) keskinimen \$50\%\$ todennäköisyydellä.
6. Muistin Vapautus	free_decade_data(), free_names()	Vapauttaa kaiken dynaamisesti varatun muistin ennen ohjelman sulkeutumista.



V. TODO / Seuraavat Askeleet

Tavoite	Kuvaus	Tila
Tulosteiden Hallinta	Kyky valita, kuinka monta nimeä generoidaan kerrallaan uuden lipun avulla (-n tai --count).	Seuraava tehtävä
Asetusten Joustavuus	Mahdollistetaan keskinimen todennäköisyyden säätäminen (esim. -m 80 80% todennäköisyys).	Suunnitteilla
Genesim-integraatio	Valmistelu genesim-projektin käyttöä varten (kirjastomoduuli).	Suunnitteilla

Yhteensopivuusratkaisu C-kielellä

1. Tilanne Windowsissa (ANSI)

- Kun datatiedostot ovat **Windows-1252**-koodauksessa (jota C-ohjelmasi Windowsissa olettaa oletuksena), merkit kuten ä ja ö luetaan oikein, koska ne ovat yhden tavun mittaisia tässä koodauksessa.
- Korjasit koodausvirheet ilman, että sinun tarvitsi muokata C-koodia Windows-spesifeillä funktioilla tai monimutkaisilla set locale-kutsuilla, jotka voisivat aiheuttaa ongelmia Linuxissa.

2. Tilanne Linuxissa (UTF-8)

- **Linuxin oletuskoodaus on yleensä UTF-8.**
- Kun siirrät **Windows-1252**-koodatut datatiedostot Linuxiin ja suoritat C-ohjelman siellä:
 - Linuxin C-kirjasto lukee tiedostot käyttämällä **UTF-8**-koodausta (tai sen paikallista oletusta).
 - **Tärkeää:** Koska **Windows-1252**-merkit (kuten ä/ö) ovat **yhden tavun** mittaisia ja sattuvat olemaan **samanlaisia** kuin UTF-8:n ensimmäinen 127 merkin joukko (eli ne eivät ole niitä ongelmallisia C3/C4-jne. tavuja), ohjelma todennäköisesti tulkitsee ne oikein näytöllä (koska useimmat Linux-päätteetkin odottavat UTF-8:aa).

Lyhyesti: Pitämällä tiedostot nyt tässä koodauksessa, vältät C-kielen koodaushaasteet (erit. set locale-funktioiden erot) käyttöjärjestelmien välillä.

Jos haluat myöhemmin siirtyä puhtaaseen **UTF-8-koodaukseen**, sinun tulee huolehtia siitä, että:

1. C-ohjelma **lukee** tiedostot erikseen eri koodauksilla molemmissa käyttöjärjestelmissä.
2. C-ohjelma **tulostaa** oikein komentoriville (Linuxissa tämä on helppoa, Windowsissa vaatii usein `_setmode( )` -kutsun konsolille).